

Introduction to Parallel Computing

a.y. 2026 – 2027

Prof. Flavio Vella

Lecture 2

Recap of Computer Architecture

Outline

Modern computer system architectures

- Von Neumann model and sequential architecture
- limits of the Von Neumann models
- techniques for reducing the latency
- superscalar architectures and ILP
- memory approaches to reduce the latency

Algorithm design, data
decomposition

Libraries and Primitives

Programming models and
implementations

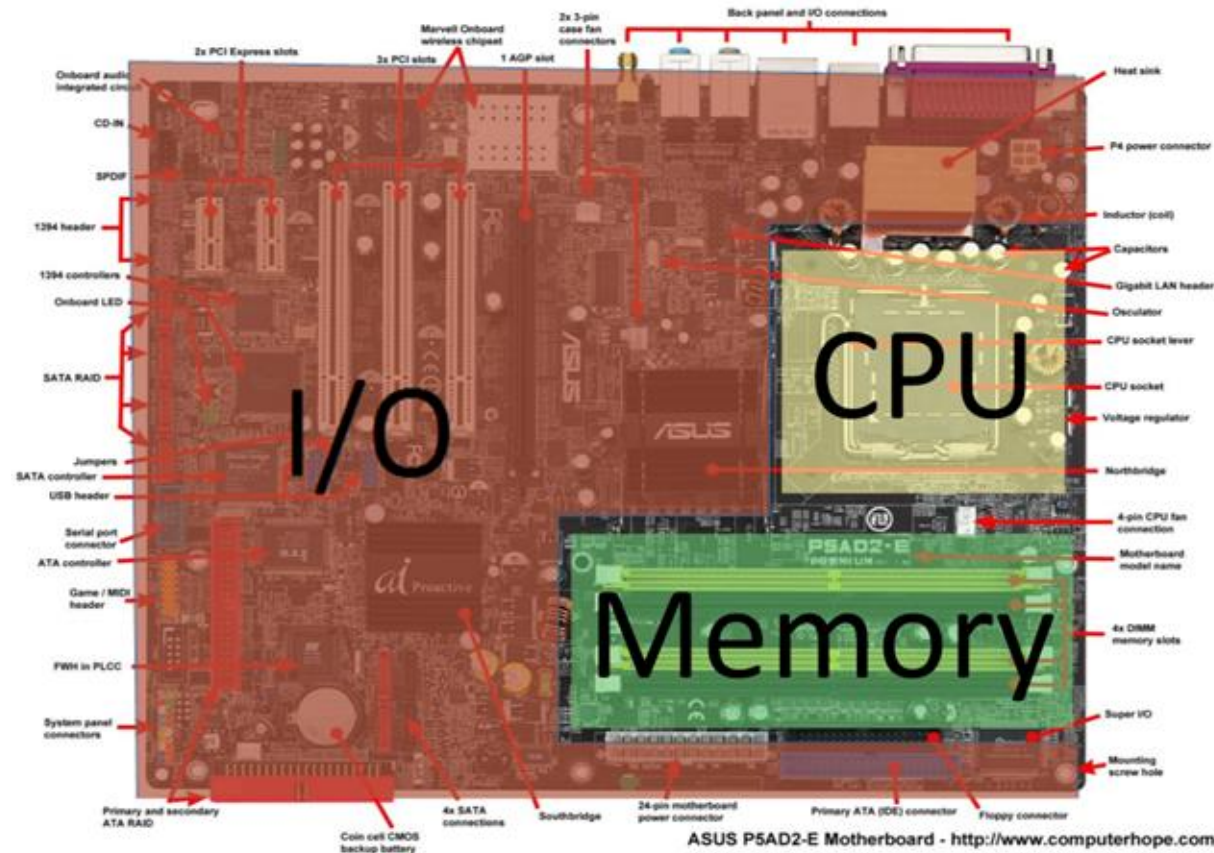
Run-time and Compilers

Architecture and Network
Interconnection

Today's computers look different



But if you look inside



Architecture model

Most of modern computers are based on the **Von Neumann architecture** (or Princeton architecture)

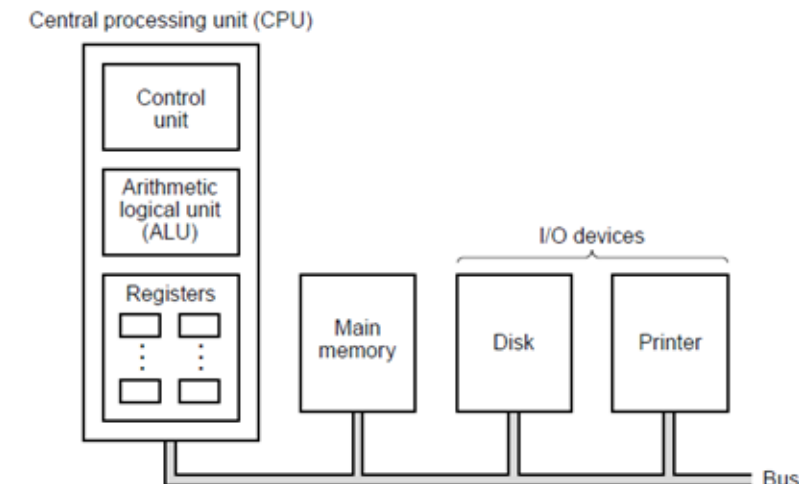
Also known as "stored-program computer" – both program instructions and data are kept in the same memory.

In the classical Von Neumann model, instructions and data share the same memory and address space and are processed by a CPU according to a sequential instruction stream



John von Neumann 1940s
(Source: LANL archives)

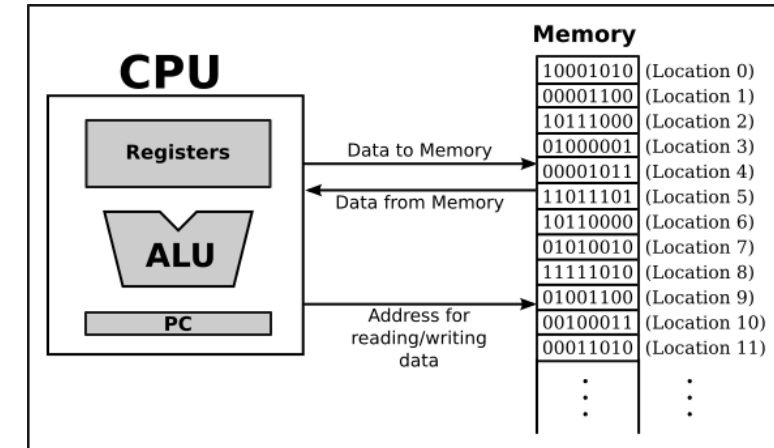
The CPU is characterized by the **FDE cycle** and **data path**



Von Neumann Architecture

Four main components:

1. Memory
2. Control Unit
3. Arithmetic Logic Unit
4. Input/Output



Random access memory (RAM) is used to store both program instructions and data

1. Program instructions are encoded operations interpreted by the processor according to the instruction set architecture (ISA)
2. Data information to be used by the program
3. From the memory perspective look the same – no from CPU side

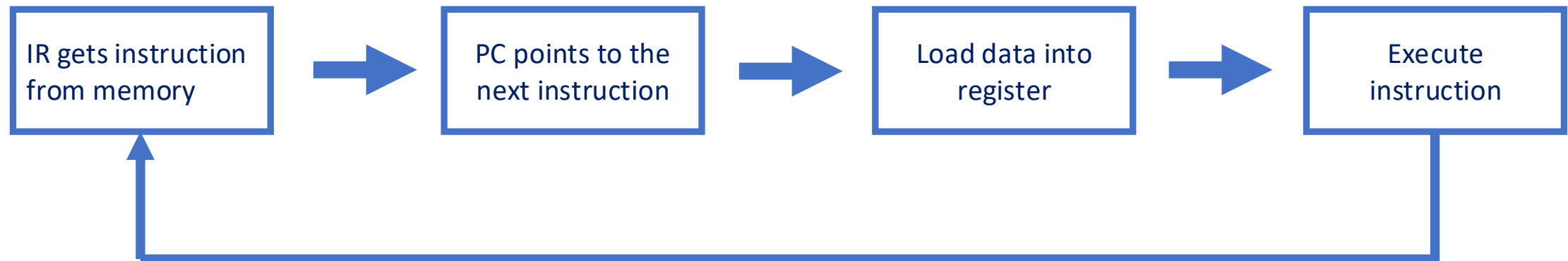
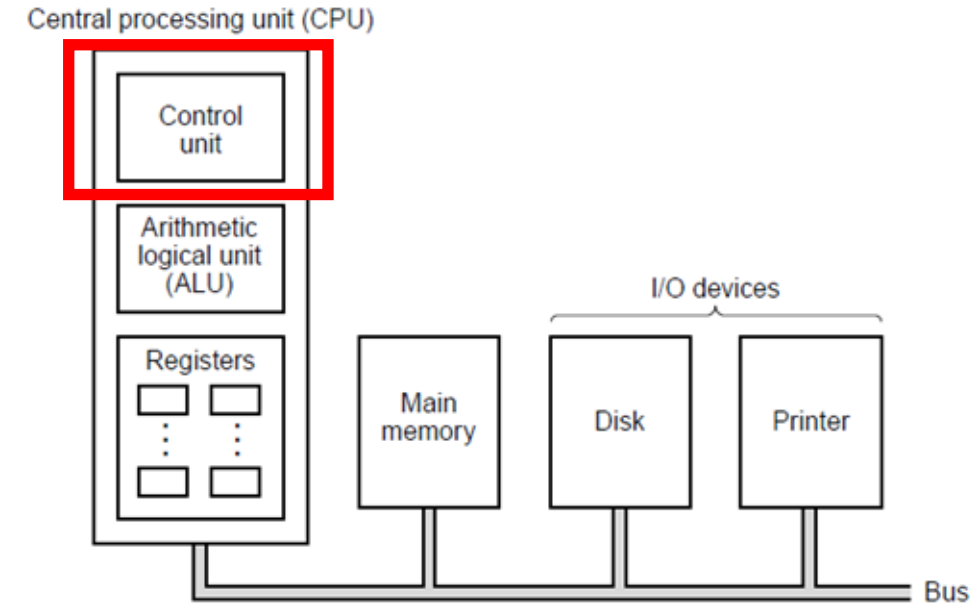
The control unit coordinates instruction fetch, decode, and execution, and controls the movement of operands between registers, memory, and execution units.

The Arithmetic Logic Unit performs basic arithmetic operations

Input/Output is the interface to the human and external environment

Fetch-Decode-Execute cycle

1. PC identifies the next instruction.
2. Instruction is fetched into the IR.
3. Instruction is decoded.
4. Required operands are obtained.
5. Operation is executed.
6. Architectural state is updated.



Data Path

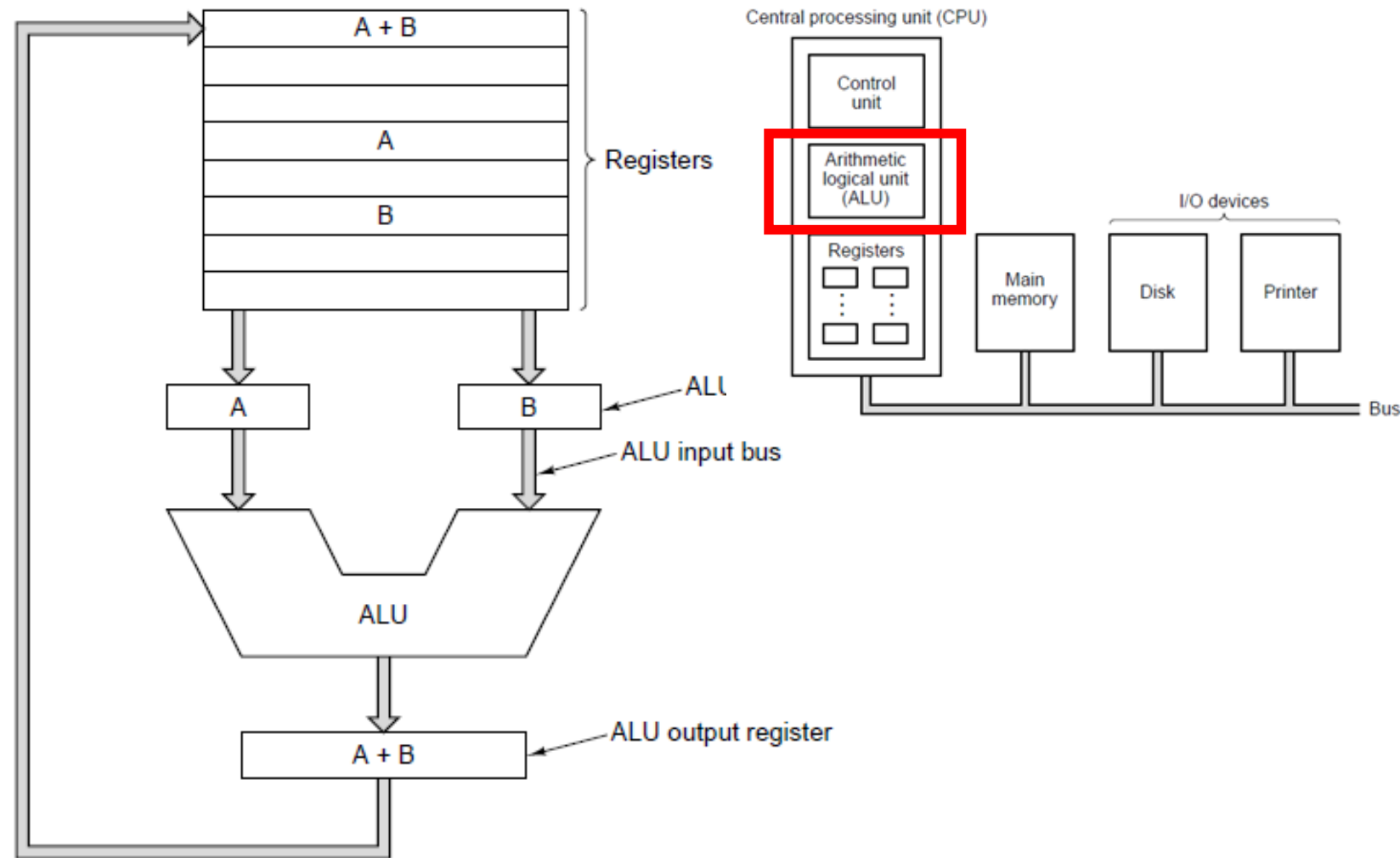
The ALU

performs arithmetic and bitwise operations on integer binary numbers

The **FPU** performs floating-point arithmetic

Data movement

Register-memory instruction
Register-register instruction



Data Path

The **ALU** performs arithmetic and bitwise operations on integer binary numbers

The **FPU** performs floating-point arithmetic

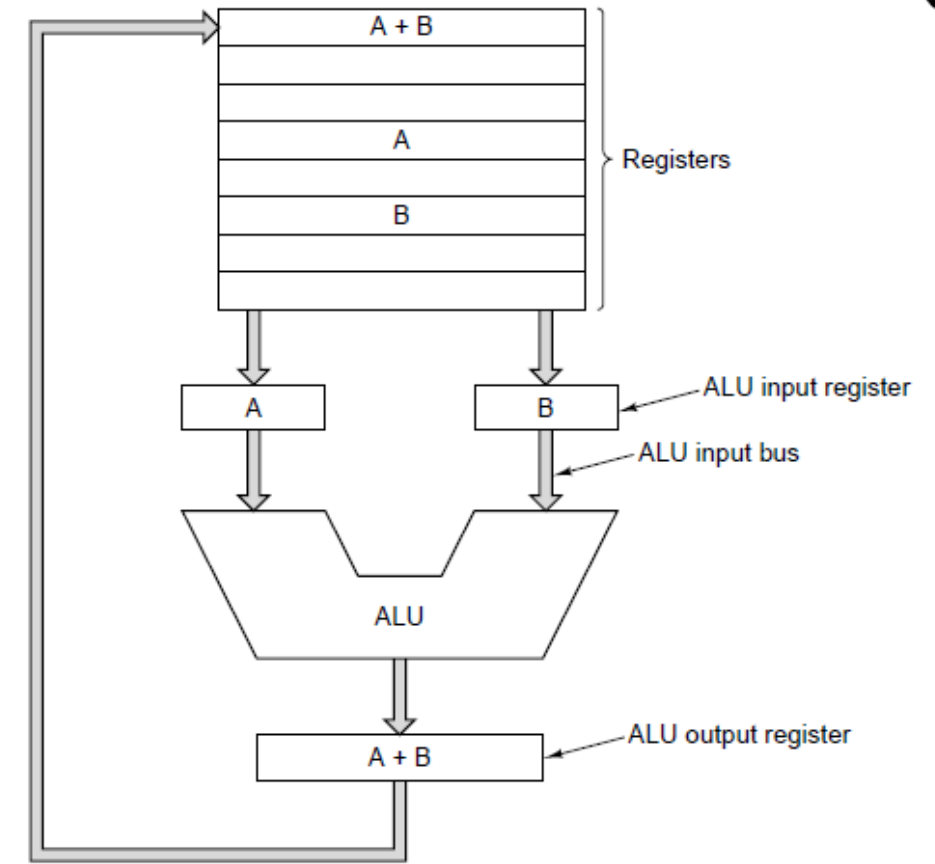
Data movement

Register-memory instruction

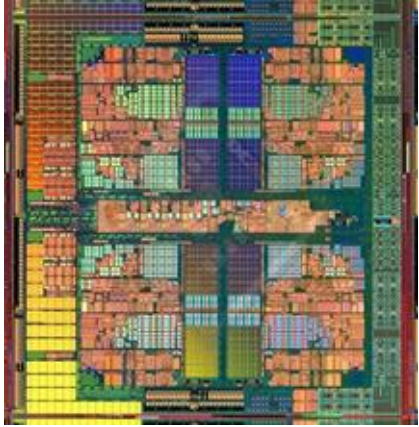
Register-register instruction

Definition:

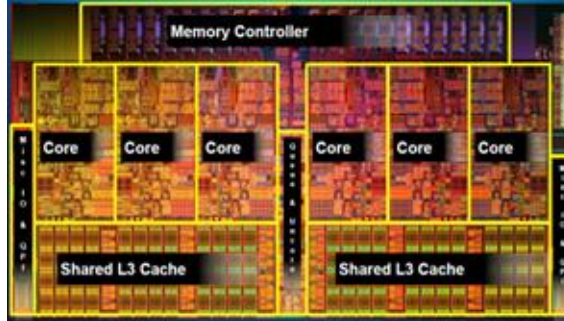
The datapath is the collection of hardware components that store, move, and operate on data, including registers, ALUs, FPUs, load/store units, and interconnections.



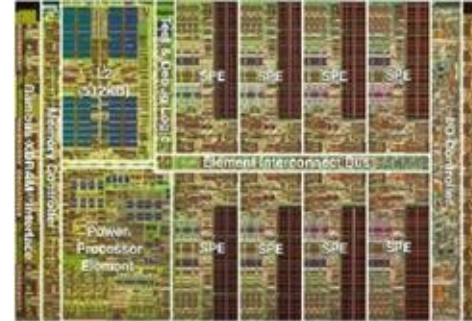
Different organization



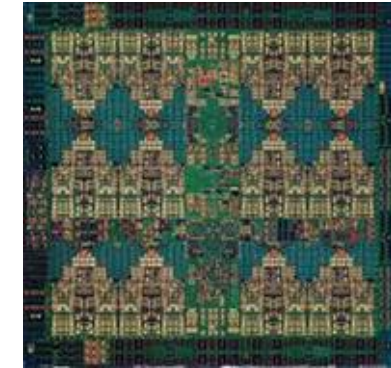
AMD Barcelona
4 cores



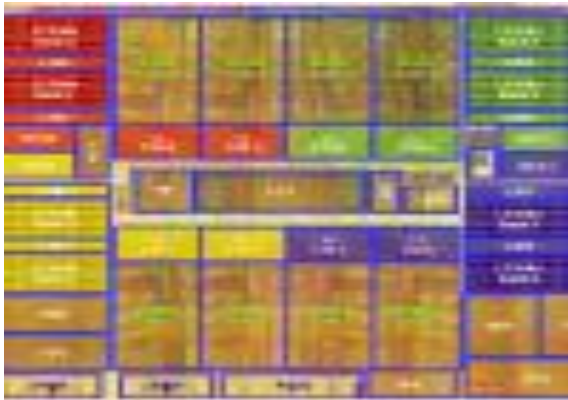
Intel Core i7
8 cores



IBM Cell BE
8+1 cores



IBM POWER9
24 cores



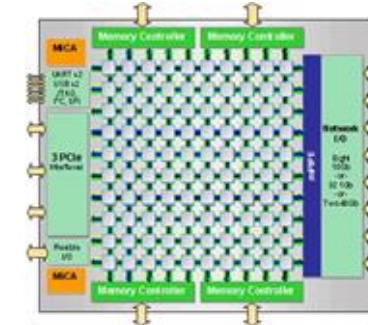
Sun Niagara II
8 cores



Nvidia Volta
5120 "cores"



Intel SCC
48 cores, networked



Tiler TILE Gx
100 cores, networked

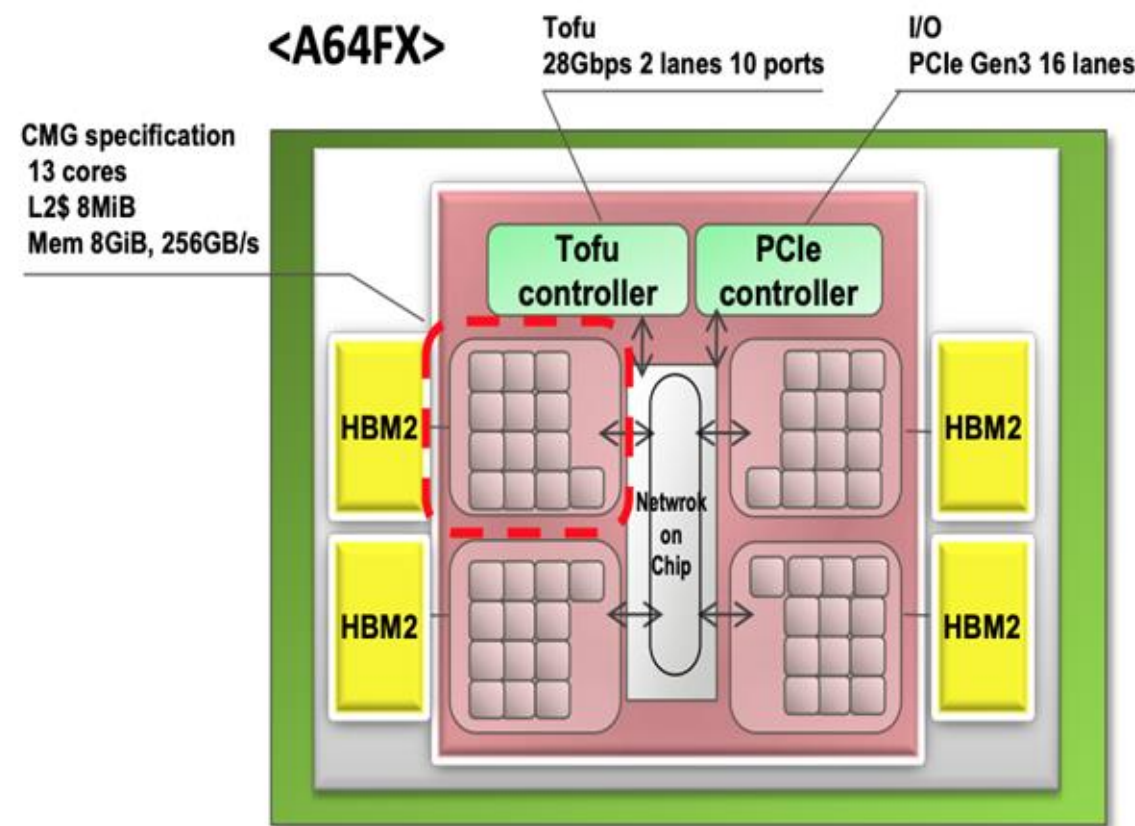
FUJITSU Processor A64FX

The A64FX processor (called A64FX, below) is a superscalar processor of the out-of-order execution type

Based on Armv8-A architecture and the Scalable Vector Extension for Armv8-A ISA

The processor integrates 52 processor cores, including 4 assistant cores

Cores are organized into 4 **Core Memory Group** (CMG)

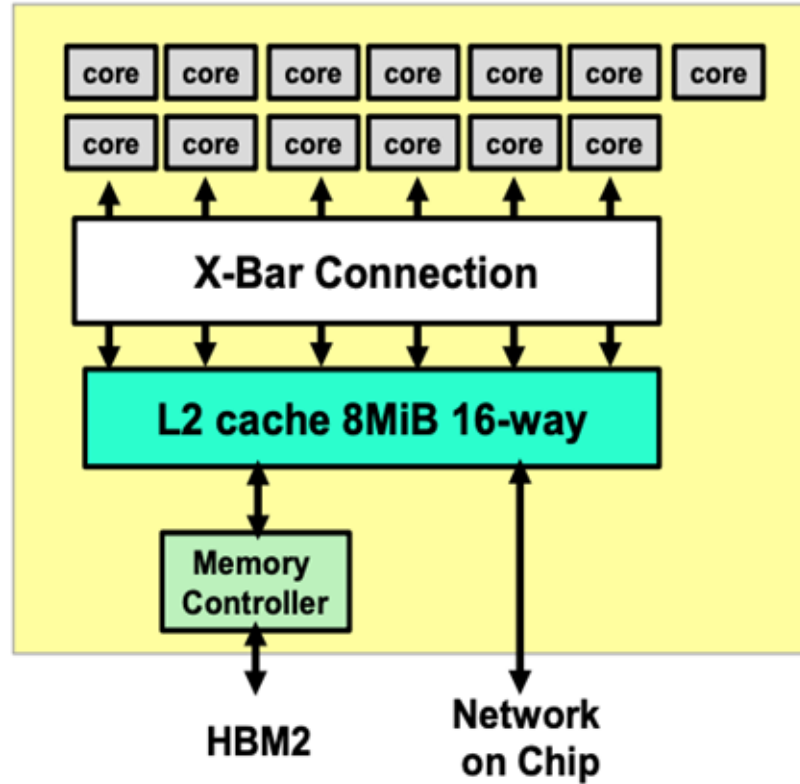


References

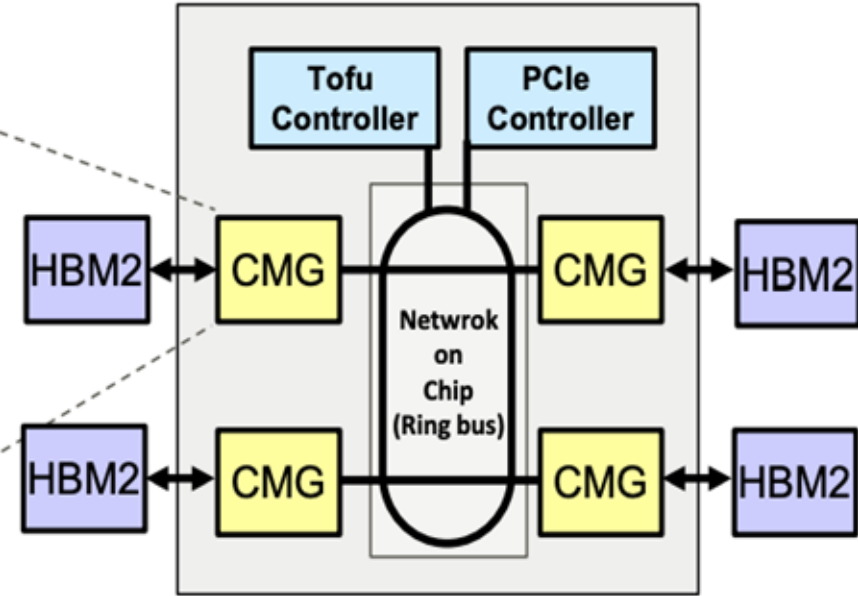
[Fujitsu High Performance CPU for the Post-K Computer a64fx_Datasheet](#)

FUJITSU Processor A64FX

CMG Configuration



A64FX Chip Configuration



GPU and accelerators

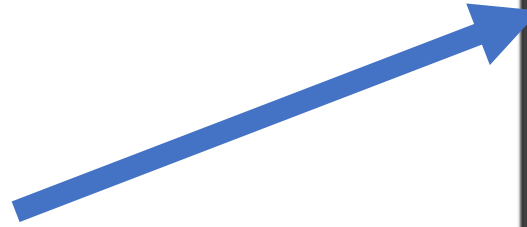


NVIDIA AMPERE GA102 GPU ARCHITECTURE

<https://www.nvidia.com/content/PDF/nvidia-ampere-ga-102-gpu-architecture-whitepaper-v2.pdf>

GPU and accelerators

NVIDIA AMPERE GA102 GPU ARCHITECTURE Detail of a Stream Multiprocessor.

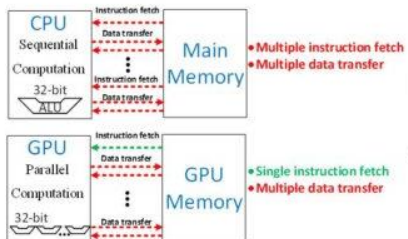


<https://www.nvidia.com/content/PDF/nvidia-ampere-ga-102-gpu-architecture-whitepaper-v2.pdf>

Many other systems

Memory Wall

Von-Neumann Architecture



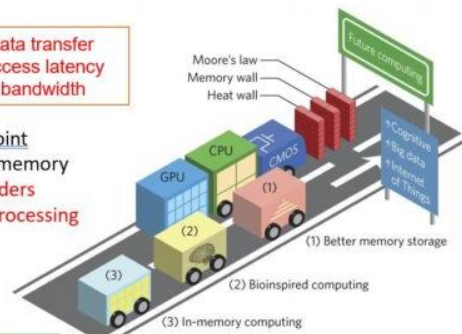
Multiplication: 3.1pJ
 Addition: 0.1pJ

On-chip cache:
 Energy: ~5pJ
 Latency: ~10ns

Off-chip memory:
 Energy: ~640pJ
 Latency: ~100ns

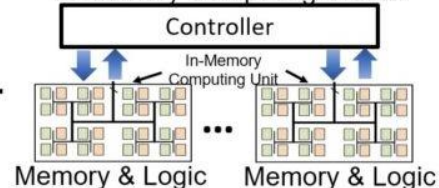
- Energy hungry data transfer
- Long memory access latency
- Limited memory bandwidth

Moving a floating point number from main memory to CPU takes **two orders more energy** than processing in CPU



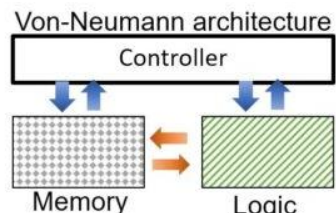
Processing-in-Memory Architecture

In-Memory Computing Cluster

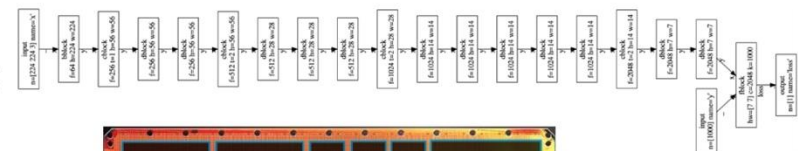


- ✓ Parallel, local data processing
- ✓ Short memory access latency
- ✓ Ultra-low energy
- ✓ Programmable, Low cost

VS.

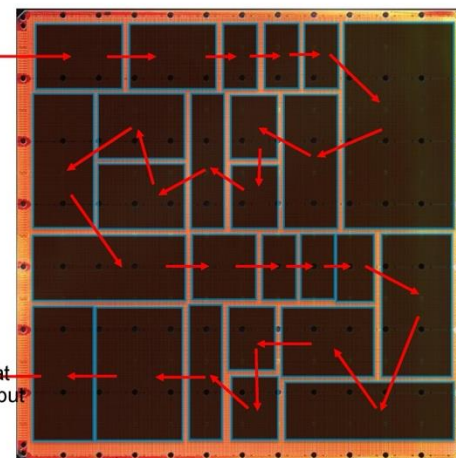


Data Streaming



Streaming data input

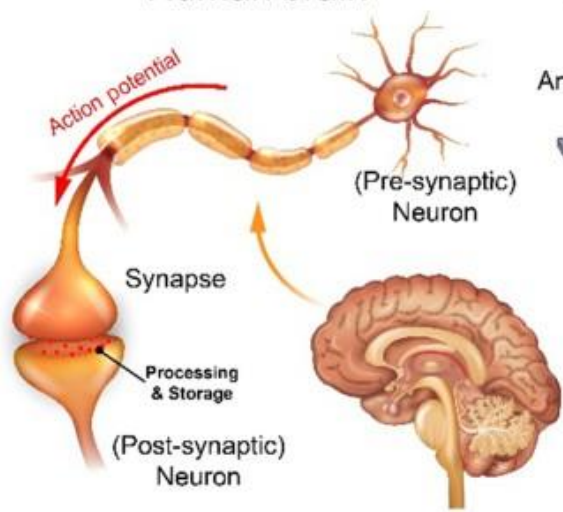
Continuously Streaming output



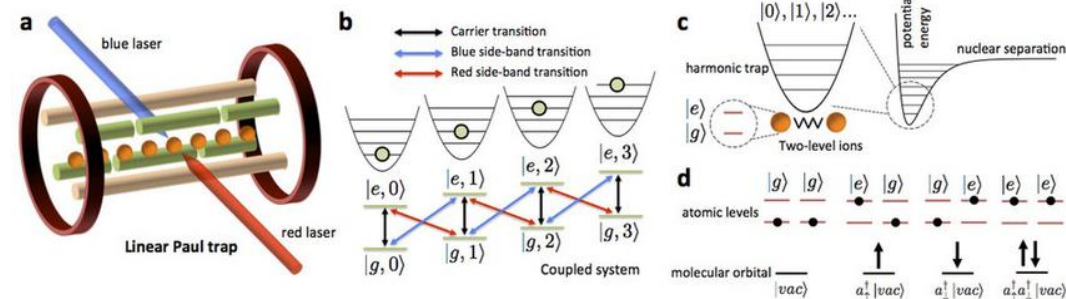
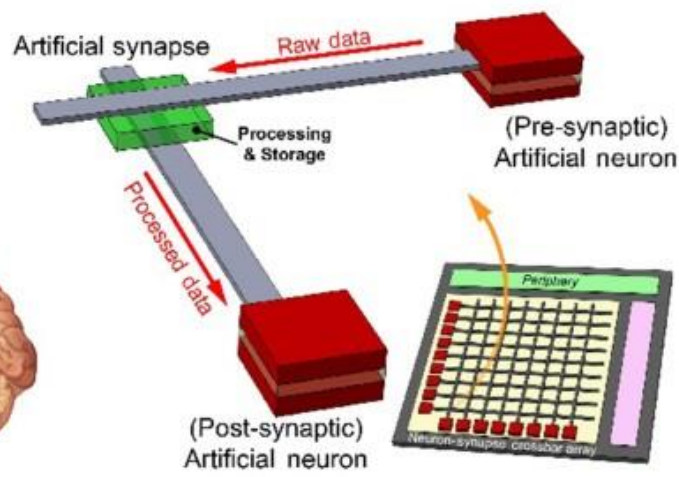
Layer-pipelined ML hardware:
 Data streams
 Weights remain stationary

Copyright © 2020 - Cerebras Systems Confidential & Proprietary - Under NDA

Human brain



Memory-centric neuromorphic chip

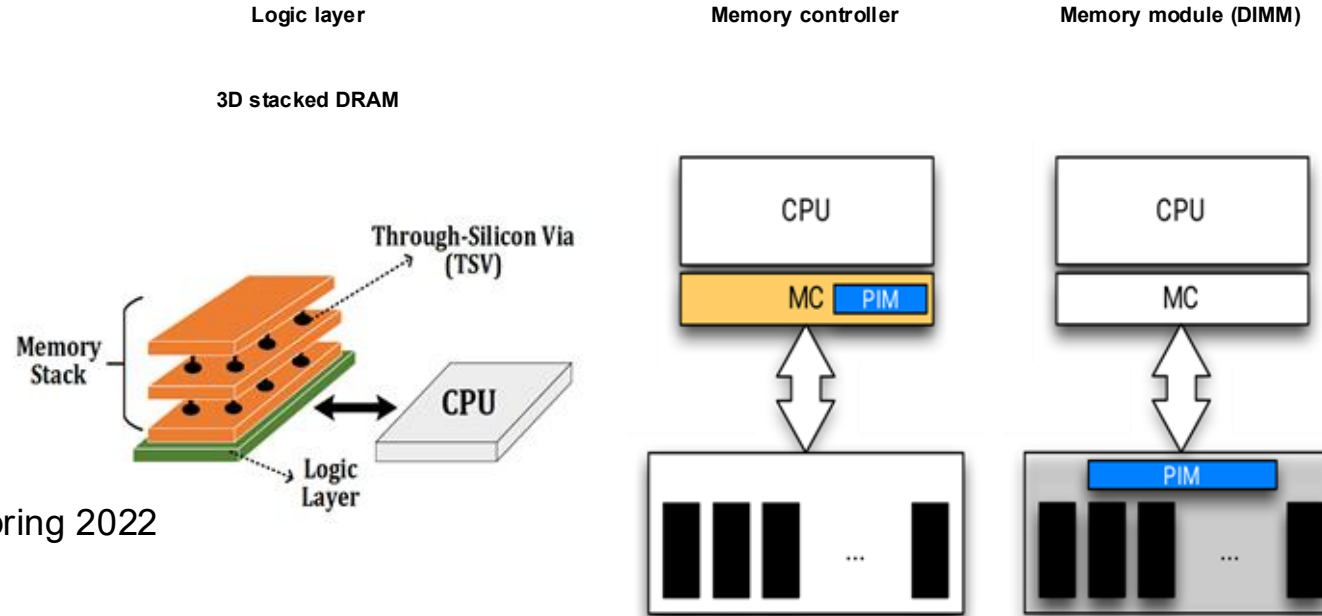


Emerging Systems

Processing Using Memory (PUM)

minimal chip modification to support Bulk Bitwise Operations[Seshadri17]

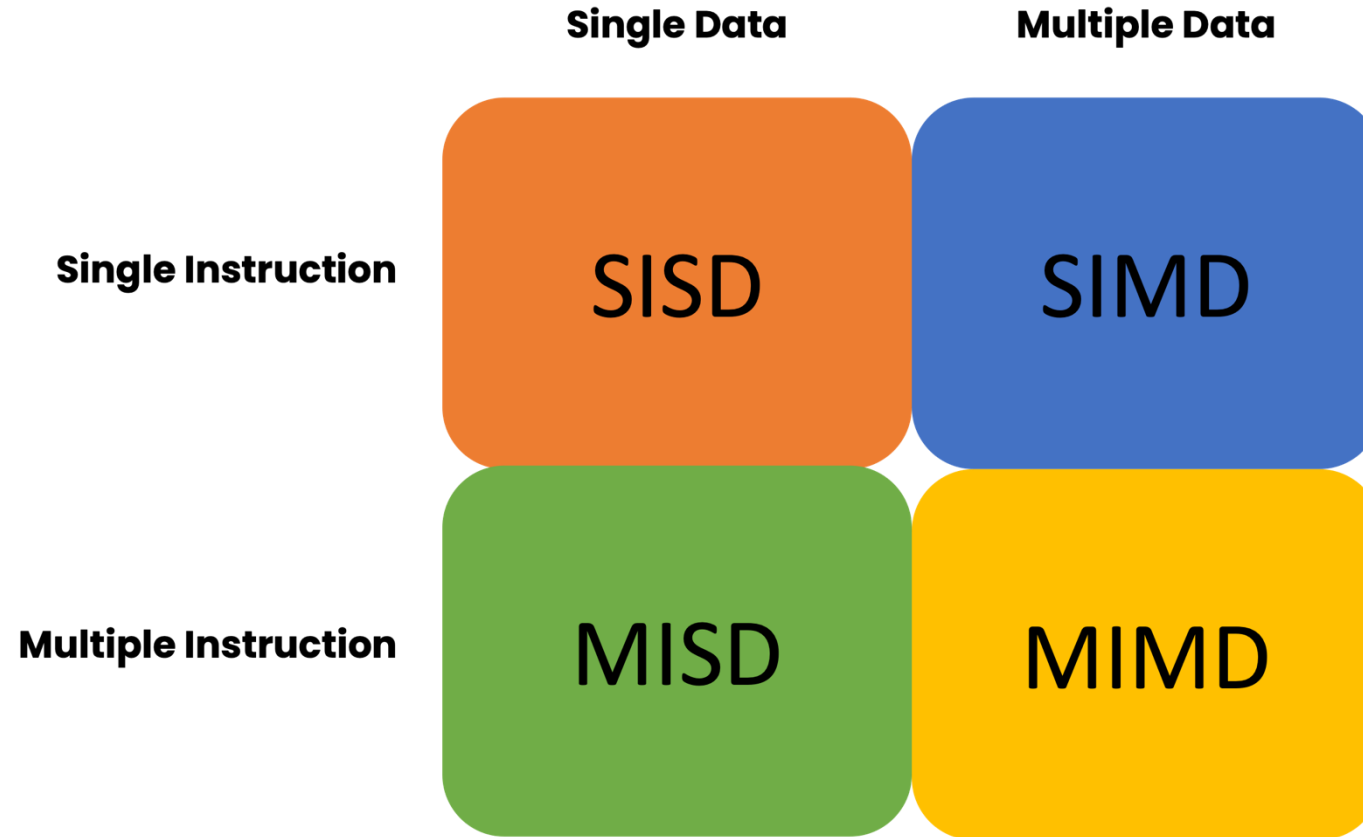
Processing near memory (PNM)example adding CU in the stack layer



J.G Luna (ETH) PIM course Spring 2022

Mutlu et al. 2023. A modern primer on processing in memory. In Emerging Computing: From Devices to Systems.
Hajinazar et al. 2021. SIMDram: a framework for bit-serial SIMD processing using DRAM. Proc. ACM ASPLOS
[Seshadri17] V. Seshadri et al., "Ambit: In-Memory Accelerator for Bulk Bitwise Operations Using Commodity DRAM Technology," 2017 50th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO), 2017, pp. 273-287.

Flynn's Taxonomy of Computers



Other Classifications

Flynn's Taxonomy of Computers

Mike Flynn, "Very High-Speed Computing Systems," Proc. of IEEE, 1966

SISD: Single instruction operates on single data element

SIMD: Single instruction operates on multiple data elements

- Array processor
- Vector processor

Single Instruction

Single Data

SISD

Multiple Data

SIMD

Multiple Instruction

MISD

MIMD

MISD: Multiple instructions operate on single data element

- "MISD is rare in general-purpose computing; practical examples are uncommon and classification is sometimes application-dependent."

MIMD: Multiple instructions operate on multiple data elements (multiple instruction streams)

- Multiprocessor
- Multithreaded processor

Flynn's Taxonomy of Computers

Mike Flynn, "Very High-Speed Computing Systems," Proc. of IEEE, 1966

SISD: Single instruction operates on single data element

SIMD: Single instruction operates on multiple data elements

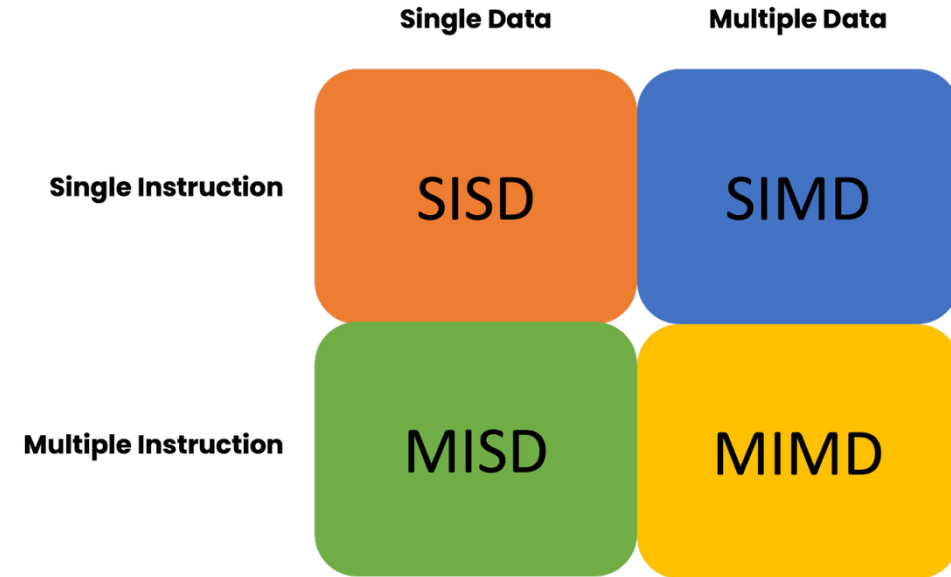
- Array processor
- Vector processor

MISD: Multiple instructions operate on single data element

- "MISD is rare in general-purpose computing; practical examples are uncommon and classification is sometimes application-dependent."

MIMD: Multiple instructions operate on multiple data elements (multiple instruction streams)

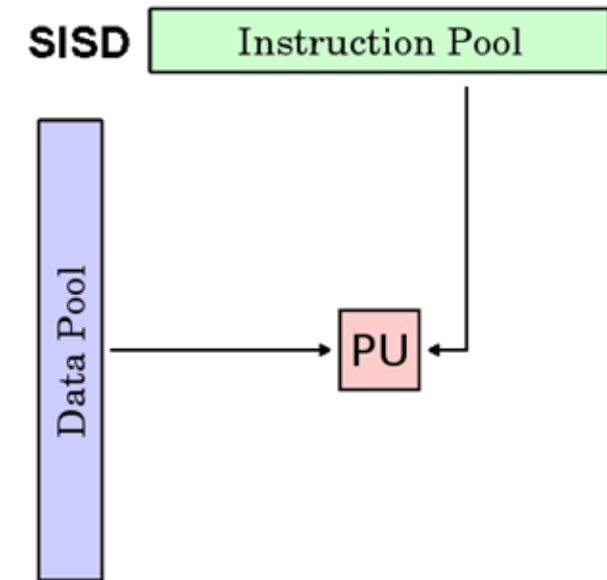
- Multiprocessor
- Multithreaded processor



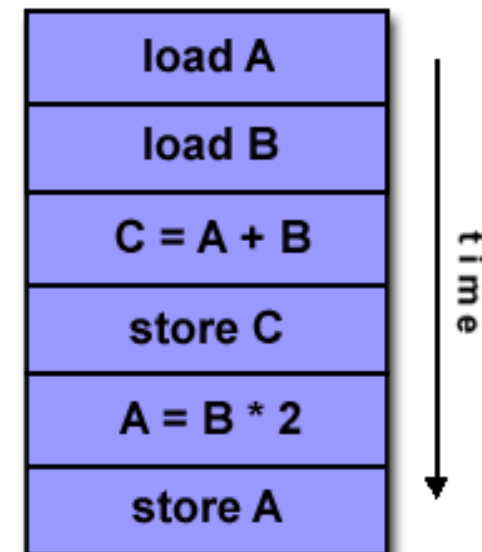
Single Instruction on a single Data

Simple serial architecture

- Single Instruction: A single logical instruction stream controls execution.
- Single Data: only one data stream is being used as input during any one clock cycle
- In-order execution (modern architectures support out-of-order)



Examples: older generation mainframes, minicomputers, workstations and single processor/core PCs



Latency and Throughput

- **Latency** is the number of clock cycles an instruction takes to have its data available for use by another instruction (**lower is better**).
- Latency also represents the cost to get the next instruction.
- **Throughput** is the number of instructions, operations, or tasks completed per unit time. (**higher is better**)

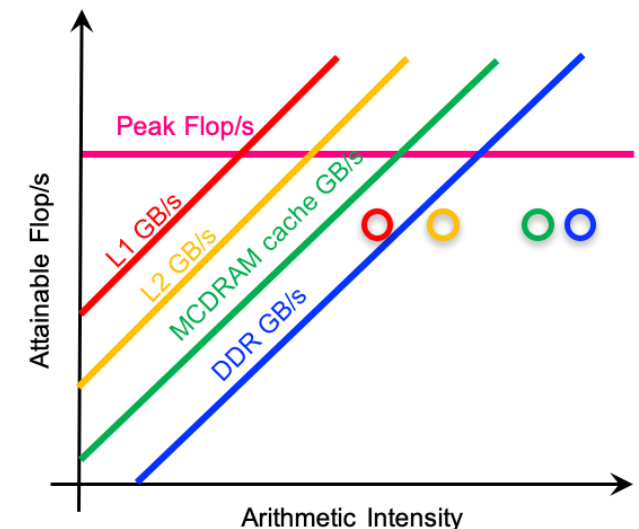
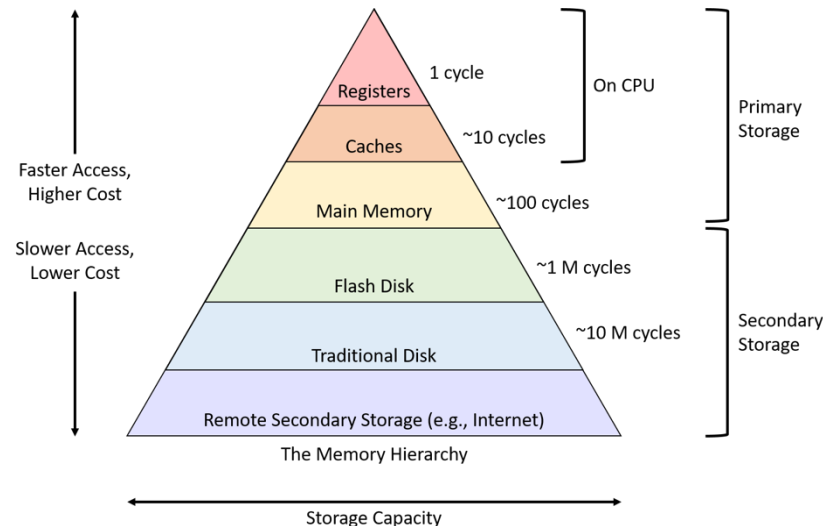
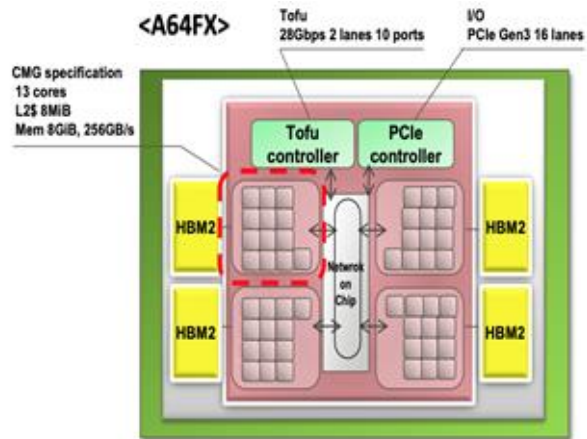
A design may aim to minimise latency, maximise throughput, or balance both.

The concepts of latency and throughput are also applied to CPU and memory.

Understanding the limits of Von Neumann architectures

Performance in VN architectures depends on many factors (**all together**)

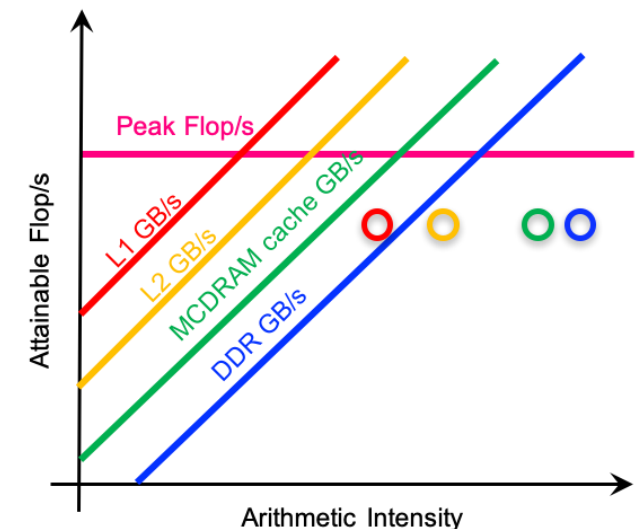
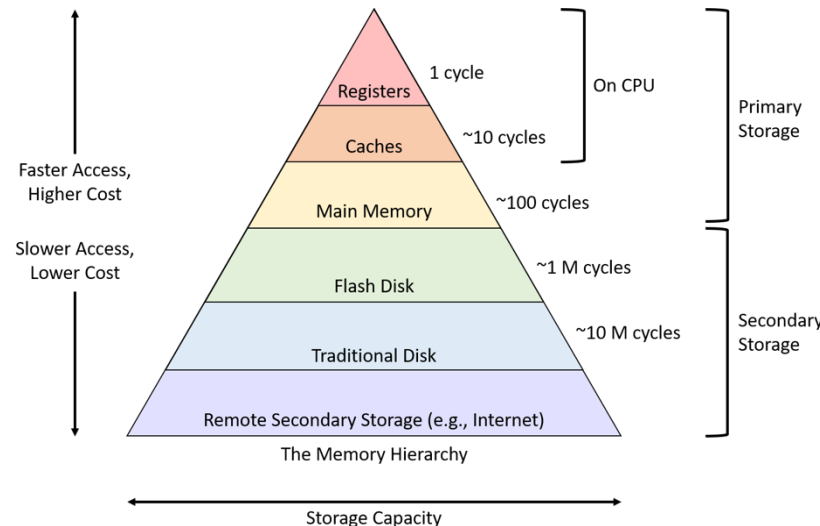
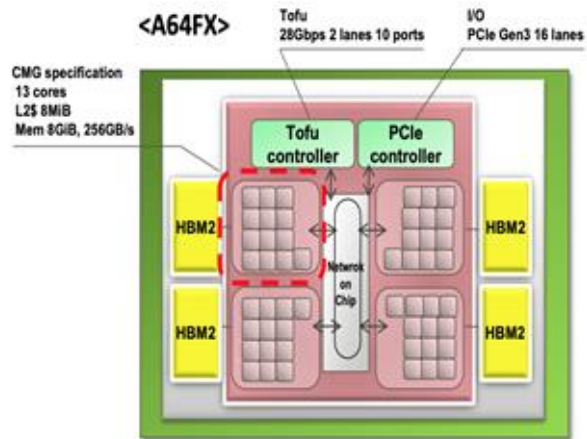
- FDE design and ISA (latency)
- ALU technology and PEs organization (math bandwidth)
- Memory sub-system (latency and memory bandwidth)
- Applications (ratio between math operations and read/write operations)



Understanding the limits of Von Neumann architectures

Performance in VN architectures depends on many factors (**all together**)

- FDE design and ISA (latency)
- ALU technology and PEs organization (math bandwidth)
- Memory sub-system (latency and memory bandwidth)
- Applications (ratio between math operations and read/write operations)

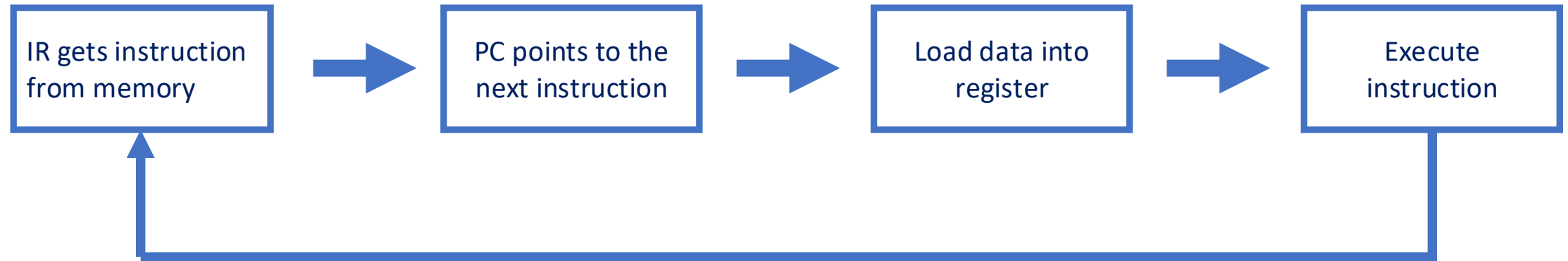


Understanding the limits of Von Neumann architectures



UNITRENTO

- FDE design and ISA (latency)



Analyze where the potential bottleneck may come from

Fetch Decode Execution Design

- Processors run at a constant rate (**clock frequency**)
- Performance also depends on the ability to execute instructions per second (MIPS) via the FDE cycle
- Different instructions may require different numbers of cycles, and the average cost is reflected in CPI.
- **For numerical workloads, FLOP/s is often used because it measures application-relevant floating-point work rather than instruction count.**

Performance depends on the task and is influenced by several factors!

- Clock cycle time: HW technology and system organisation
- Clock cycle per instruction (**CPI**): system organisation and **ISA**
- Instruction count (**IC**): ISA, compiler technology (program used for the benchmark)

$$CPU\ time = IC \times Cycles\ per\ instruction \times Clock\ cycle\ time$$

Implicit Parallelism: Trends in Microprocessor Architectures



UNITRENTO

Implicit parallelism is parallel execution extracted automatically by the compiler and/or hardware without the programmer explicitly creating parallel tasks or threads. It contrasts with explicit parallelism, where the programmer specifically indicates parallel sections in the code.

- Historically, increasing clock frequency was a major source of performance improvement, but power and thermal constraints limited continued frequency scaling.
- Higher levels of device integration have made available many transistors.
- the question of how best to utilise these resources is an important one

Parallelism on the Hardware Level

- Current processors use these resources in multiple functional units and execute multiple operations in the same cycle (Instruction Level Parallelism and Superscalar Architectures).
- The way in which these instructions are selected and executed provides different architectures (out-of-order execution and pipelining).
- Vectorization

Instruction Level Parallelism (ILP)

Definition

Instruction-level parallelism (ILP) is the ability to execute multiple independent instructions from a single instruction stream concurrently

- Introduction of an execution unit (also called a functional unit)

It is responsible for performing the operations (load and store and arithmetic operations) as instructed by the computer program.

It may have its own internal control sequence unit, which is not to be confused with the CPU's main control unit, and some registers.

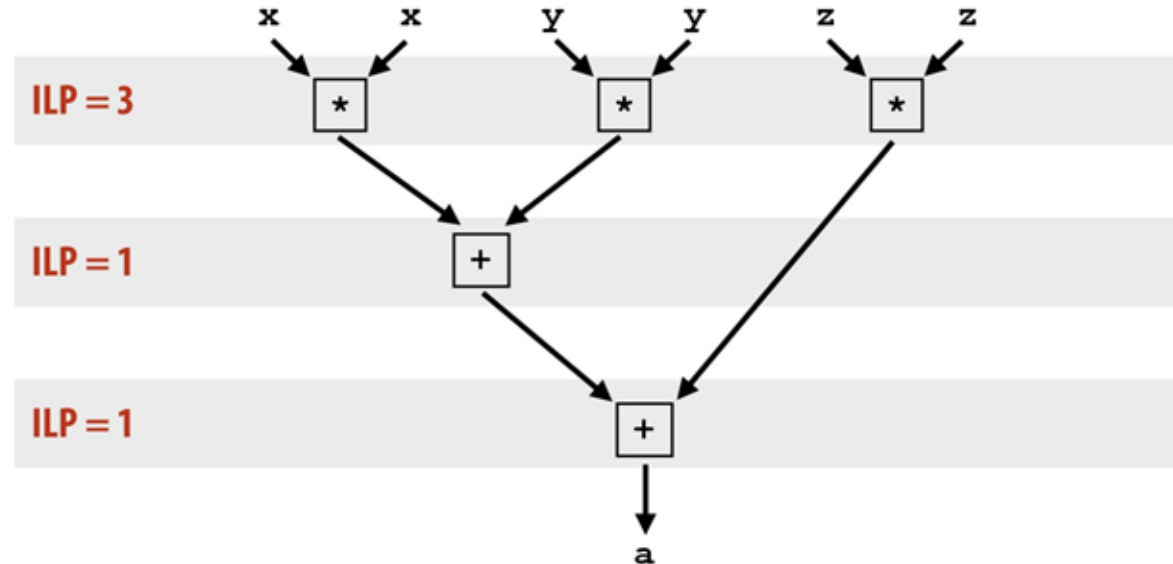
- **Components are** arithmetic logic unit (ALU), address generation unit (AGU), floating-point unit (FPU), load-store unit (LSU), branch execution unit (BEU);
- Superscalar architectures have many EUs.

Example of Superscalar execution

A superscalar processor is a CPU that implements a form of parallelism called instruction-level parallelism within a single processor.

It can find instruction dependencies

$$a = x * x + y * y + z * z$$



Source Parallel Computer Architecture and Programming, Carnegie Mellon.

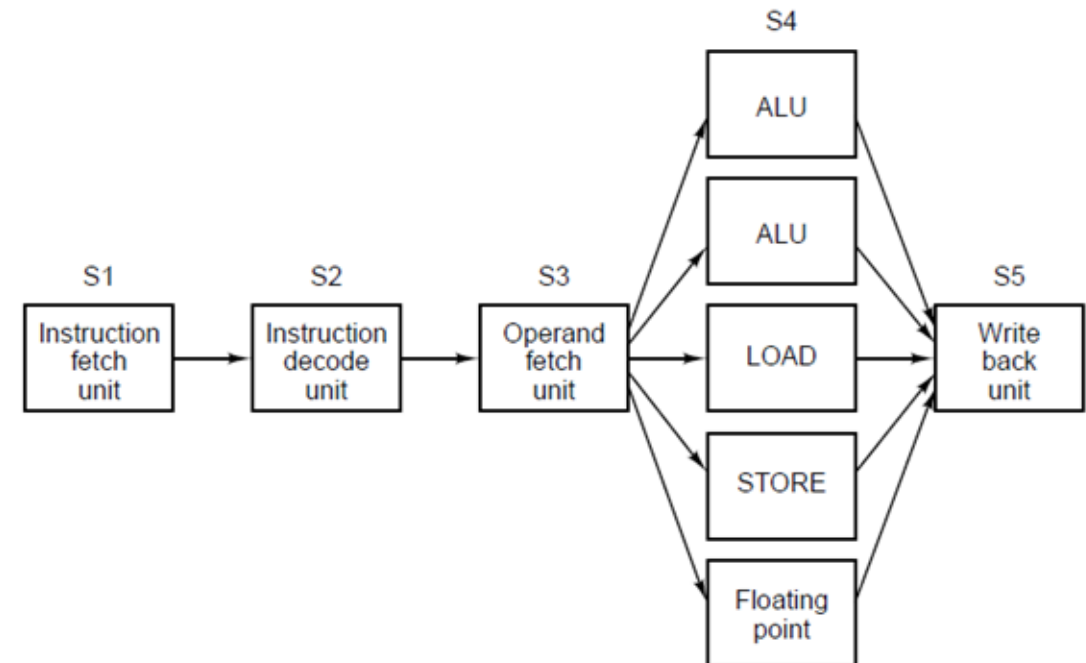
Dependent instructions

mul	r1, r0, r0	←
mul	r1, r1, r1	←
st	r1, mem[r2]	←
...		
add	r0, r0, r3	←
add	r1, r4, r5	←
...		
...		

Independent instructions

Modern architectures

- In Flynn's taxonomy, a single-core superscalar processor is classified as an SISD.
- Superscalar design can be combined with other performance enhancement technique:
 - **Speculative execution**: predict results to continue execution
 - **Out-of-Order (OoO) execution**: potentially change execution order of instructions
 - **Pipeline**
 - **Vectorization (SIMD approach)**



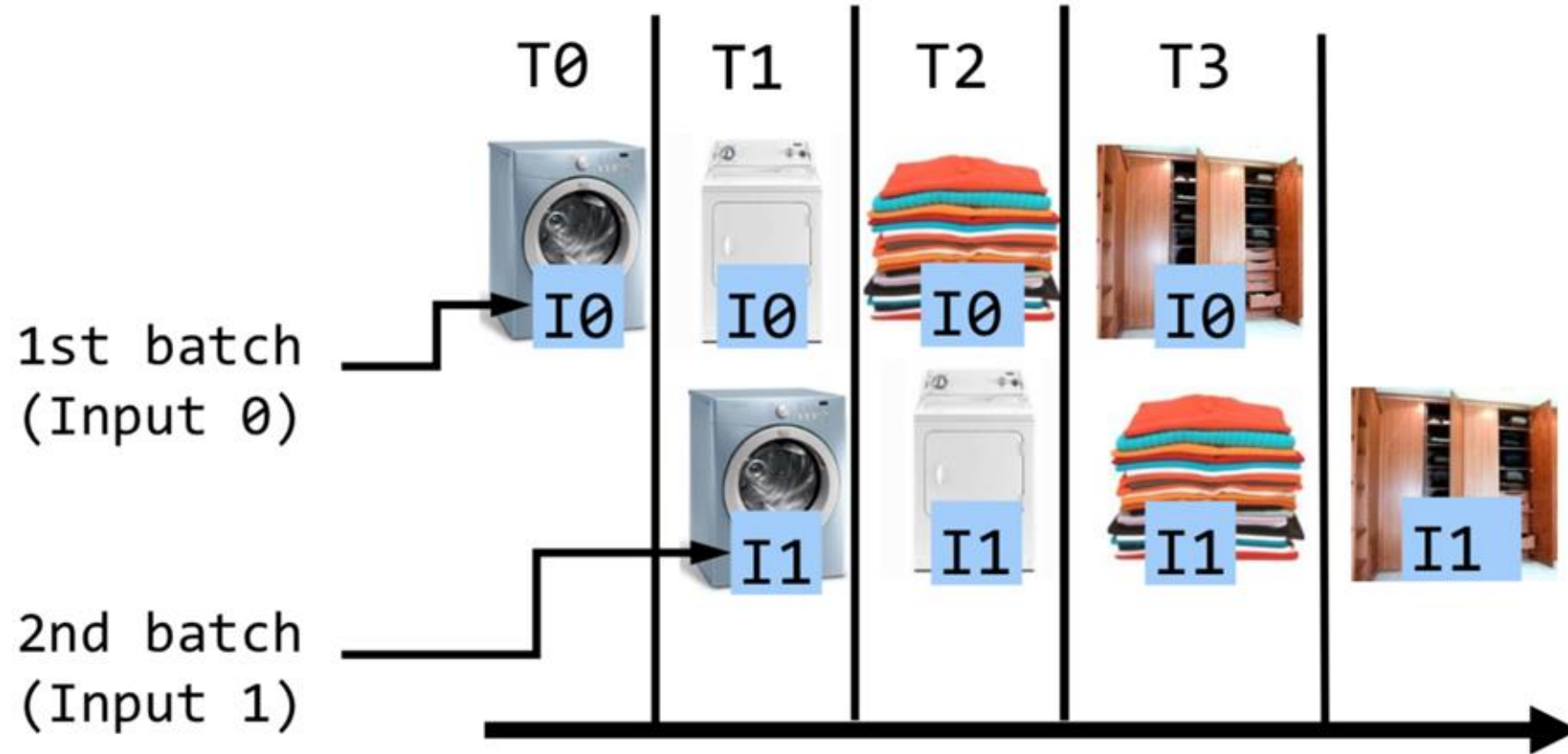
Pipeline



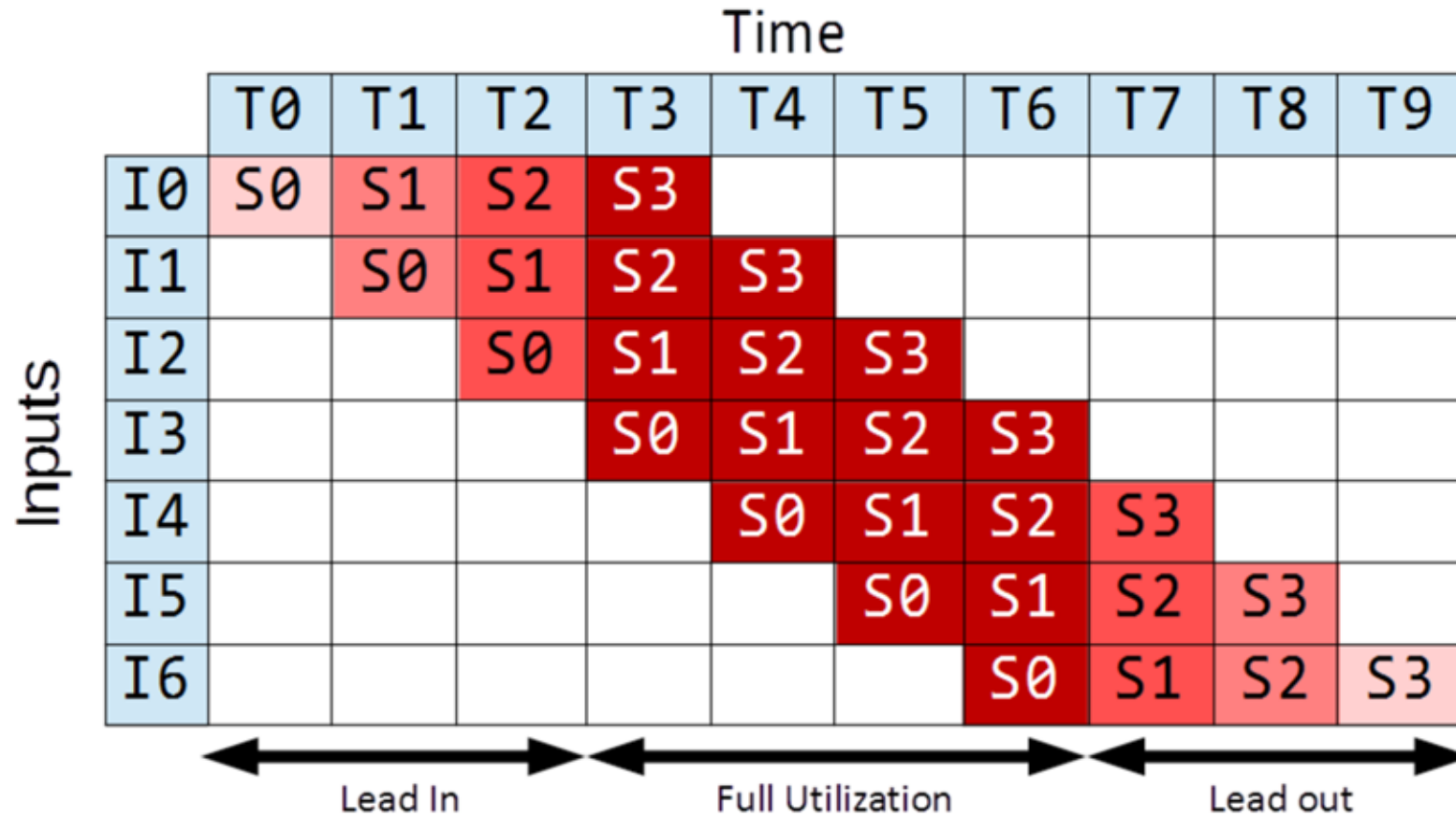
The goal is **to reduce the latency and maximize the use of the resource**

Washing clothes example (credits ETH Introduction to Parallel Programming)

Pipeline



Inputs are instruction streams



Balanced Pipeline: all the steps require the same time (practically never happens)

Throughput bound = $1/\max(\text{computation time (stages)})$

Latency bound = $\#stages * \max(\text{computation time (stages)})$

Pipeline



UNITRENTO

Pipelines are usually unbalanced



Takes 5 seconds. Stage: "w" stands for Washer.



Takes 10 seconds. Stage "d" stands for Dryer.



Takes 5 seconds. Stage "f" stands for Folding.



Takes 10 seconds. Stage "c" stands for Closet.

Pipeline

Time (s) Load #	0	5	10	15	20	25	30	35	40	45	50	55	60	65	70
Load 1	w	d	d	f	c	c									
Load 2		w	—	d	d	f	c	c							
Load 3			w	—	—	d	d	f	c	c					
Load 4				w	—	—	—	d	d	f	c	c			
Load 5					w	—	—	—	—	d	d	f	c	c	

- We are assuming 5 washing loads.
- The total time for all is **70 secs!**
- **Latency issue**
- **Solution:** one approach is to make each stage take as much time as the longest one

Pipeline

Pipelines are usually unbalanced



Takes 10 seconds.



Takes 10 seconds (as before)



Takes 10 seconds



Takes 10 seconds (as before)

Pipeline

Pipelines are usually unbalanced



Takes 10 seconds.



Takes 10 seconds (as before)



Takes 10 seconds



Takes 10 seconds (as before)

We are balancing it!!!

Pipeline

The total time for all is **80 secs!**

Latency is bound at 40 seconds for each Load.

Throughput 1 load / 10 seconds, so about 6 loads / minute.

Can we improve latency and throughput at the same time?

Time (s) Load #	0	10	20	30	40	50	60	70	80	90	100
Load 1	w	d	f	c							
Load 2		w	d	f	c						
Load 3			w	d	f	c					
Load 4				w	d	f	c				
Load 5					w	d	f	c			

Pipeline

Increase the FU



Takes 10 seconds



Adding 2 dryers working in a row. The first dryer is referred to as d1 and takes 4 seconds, the second as d2 and takes 6 sec.



Takes 5 seconds



Adding 2 closets working in a row. The first closet is referred to as c1 and takes 4 seconds, the second as c2 and takes 6 sec.

Pipeline

Bounding of the latency



6 seconds



Each of d1 and d2 dryers take 6 seconds



6 seconds



Each of c1 and c2 closets now take 6 seconds

Pipeline

The total time for all is 60 secs!

Latency is bound at 36 (6*6) seconds for each Load.

Throughput 1 load / 6 seconds = ~ 10 loads / minute.

Time (s) Load #	0	6	12	18	24	30	36	42	48	54	60	110
Load 1	w	d1	d2	f	c1	c2						
Load 2		w	d1	d2	f	c1	c2					
Load 3			w	d1	d2	f	c1	c2				
Load 4				w	d1	d2	f	c1	c2			
Load 5					w	d1	d2	f	c1	c2		

Pipeline

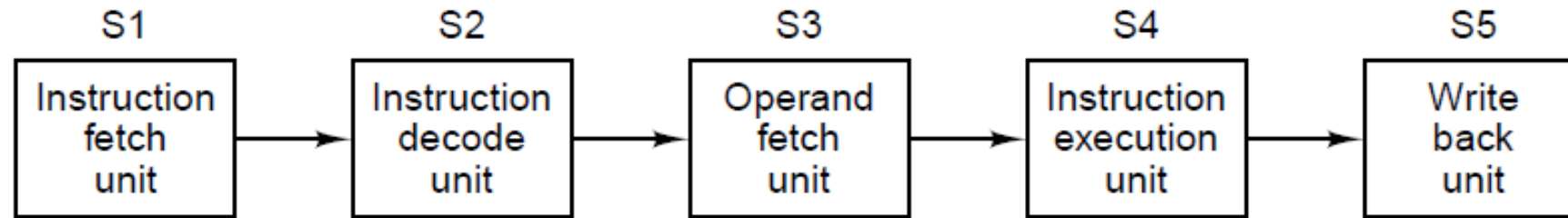


UNITRENTO

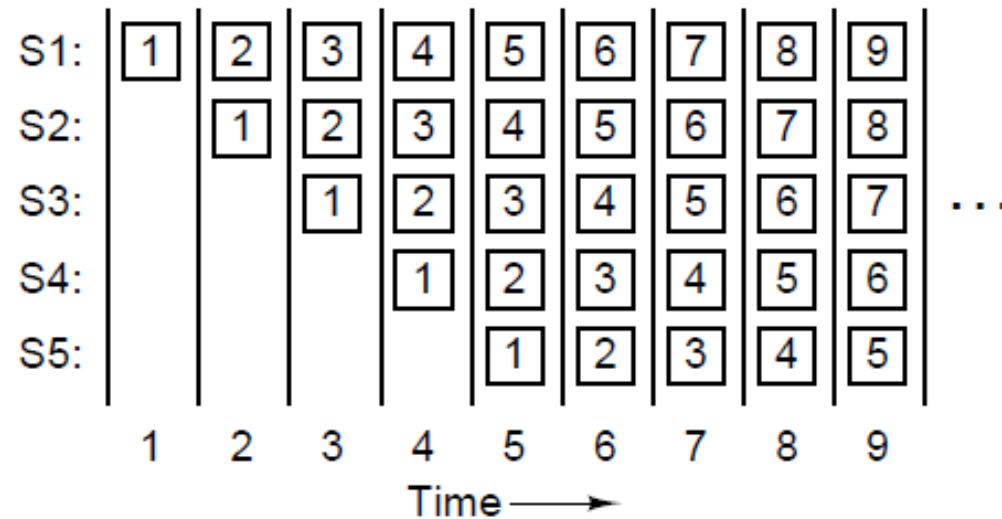
- **Fetching** of instructions from memory is a major bottleneck in instruction execution speed.
- Instructions can be stored in a set of registers called the **prefetch buffer**.
- **Execution** is divided into two parts: fetching and actual execution;
- Instead of dividing instruction execution into only two parts, it is often divided into many parts, each one handled by a dedicated piece of hardware, all of which can run in parallel.

Pipeline

A simple 5 stage pipeline



(a)



(b)

References



UNITRENTO

An Introduction to Parallel Programming (Pacheco). Chapter 2 “Parallel hardware and parallel software.” Sections 2.1 – 2.2 – 2.3.